

Verificación Computacional: Jacobiano Analítico 2R

Este script correlaciona el modelo matemático simbólico con la evaluación numérica, mapeo de velocidades y cálculo de rango (para validación de singularidades)[cite: 11]. *Dependencias requeridas: Python 3 y Numpy.*

```
import numpy as np

def jacobian_2r(q, a1=1.0, a2=1.0):
    """
    Retorna el sub-jacobiano de traslacion
    2x2 para brazo 2R
    """
    q1, q2 = q

    s1 = np.sin(q1)
    c1 = np.cos(q1)
    s12 = np.sin(q1 + q2)
    c12 = np.cos(q1 + q2)

    # Ensamblaje del modelo simbolico derivado en clase
    return np.array([
        [-a1*s1 - a2*s12, -a2*s12],
        [ a1*c1 + a2*c12, a2*c12]
    ], dtype=float)

# =====
# 1. PRUEBA DE MAPEO DE VELOCIDAD
# =====

# q1=0, q2=90 grados
q = np.array([0.0, np.pi/2])

# qdot: velocidades articulares comandadas
qd = np.array([1.0, -1.0])

J = jacobian_2r(q)
pd = J @ qd          # Map: J * q_dot

print("--- Configuracion Regular ---")
print("Matriz J =\n", np.round(J, 4))
print("Vel. Cartesiana (p_dot) =",
      np.round(pd, 4))
print("Rango(J) =", np.linalg.matrix_rank(J))

# =====
# 2. PRUEBA DE SINGULARIDAD (Brazo estirado)
# =====

# q2 = 0 es configuracion singular analitica
q_sing = np.array([0.0, 0.0])
J_sing = jacobian_2r(q_sing)

print("\n--- Configuracion Singular ---")
print("Matriz J_sing =\n", np.round(J_sing, 4))
print("Rango(J_sing) =", np.linalg.matrix_rank(J_sing))
```

Salida Esperada y Correspondencia

Configuración Regular: La salida de $\dot{p}$ será $[0. \ 1.]$, confirmando la cancelación de la traslación en X (rango = 2, control cartesiano completo).

Configuración Singular: La matriz tendrá columnas dependientes $\begin{bmatrix} 0. & 0. \\ 2. & 1. \end{bmatrix}$, y el rango caerá a 1, comprobando matemáticamente que el robot perdió movilidad instantánea independiente en una dirección[cite: 11].